SQL

DATA TYPES

1.Numeric types- used for storing numbers

- INTEGER/INT- a 4-byte signed integer (-2,147,483,648 to 2,147,483,647)
- DECIMAL(p, s) / NUMERIC(p, s) - Exact decimal with user-defined precision (p) and scale (s)
- SERIAL - Auto-incrementing 4-byte integer

2.Character types- used for storing texts or strings

- CHAR(n)/- fixed length string
- VARCHAR(n)- variable-length string with limit n
- TEXT - variable unlimited length string

3. Boolean type

- BOOLEAN - stores TRUE FALSE or NULL

4.date and time type

- Date – calendar date (year month day)
- TIMESTAMP- date and time with no time zone
- TIMESTAMPZ – date and time with time zone
- INTERVAL - span of time (a span of 2 days or a span of last 6 months)

CONSTRAINTS

1.PIMARY KEY – uniquely identifies each row. It combines NOT NULL and UNIQUE

2.FOREIGN KEY – establishes a relationship between two tables by referencing a primary key in another table.

3.NOT NULL – prevents null empty values in a column.

4. UNIQUE – ensures all values in a column are distinct. A table can have multiple unique constraints.

Creating a table : The following SQL statement will create a table named cars in the PostgreSQL database.

```
CREATE TABLE cars (
    brand VARCHAR(255),
    model VARCHAR(255),
    year INT
);
```

Inserting values: values are inserted into a table using the following statements

```
INSERT INTO cars (brand, model, year)
VALUES ('Ford', 'Mustang', 1964);
```

Fetching data: to retrieve data from a table the select statement is used as follows

```
<postgres> Script    *<postgres> Script-2
SELECT brand, year FROM cars;
```

Update columns: The UPDATE statement is used to modify the values in existing records in a table.

```
UPDATE cars
SET color = 'red'
WHERE brand = 'Volvo';
```

Deleting columns: To remove a column from a table, we have to use the ALTER TABLE statement.

```
ALTER TABLE cars
DROP COLUMN color;
```

Deleting values in a column: The DELETE statement is used to delete existing records in a table.

```
DELETE FROM cars
WHERE brand = 'Volvo';
```

Deleting entire table: The DROP TABLE statement is used to drop an existing table in a database.

```
DROP TABLE cars;
```

TRUNCATE: this deletes all rows(values in a column) from a table without deleting the columns.

```
TRUNCATE TABLE users;
```

GROUP BY - groups rows that have the same values into summary rows, like "find the number of customers in each country". It is often used with aggregate functions like COUNT(), MAX(), MIN(), SUM(), AVG() to group the result-set by one or more columns.

```
SELECT COUNT(customer_id), country
FROM customers
GROUP BY country;
```

LIKE

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

There are two wildcards often used in conjunction with the LIKE operator:

- % The percent sign represents zero, one, or multiple characters
- _ The underscore sign represents one, single character

1. Find cars with names starting with "Toyota"

```
SELECT * FROM cars
WHERE car_name LIKE 'Toyota%';
```

% is a **wildcard** that matches any number of characters. Toyota% matches: "Toyota Premio", "Toyota Vitz", etc.

2. Find cars with names ending in "Fit"

```
SELECT * FROM cars
WHERE car_name LIKE '%Fit';
```

3. Find cars that contain the word "Explorer"

```
SELECT * FROM cars
 WHERE car_name LIKE '%Explorer%';
```

4.Case-insensitive search using ILIKE

```
SELECT *
 FROM cars
 WHERE car_name ILIKE 'toyota%';
```

5. Pattern Matching with Underscore _ The underscore _ matches exactly one character.

```
SELECT *
 FROM cars
 WHERE model LIKE '_X';
```

Matches any model name that has two characters, the second being X.

JOIN

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.

1.INNER JOIN - The INNER JOIN keyword selects records that have matching values in both tables.

```
SELECT c.car_name, c.type, cs.stock
FROM cars c
INNER JOIN car_stock cs ON c.id = cs.car_id;
```

2.LEFT JOIN - The LEFT JOIN keyword selects ALL records from the left table, and the matching records from the right table. The result is 0 records from the right side if there is no match.

3.RIGHT JOIN - The RIGHT JOIN keyword selects ALL records from the right table, and the matching records from the left table. The result is 0 records from the left side if there is no match.

4.FULL JOIN - The FULL JOIN keyword selects ALL records from both tables, even if there is not a match. For rows with a match the values from both tables are available, if there is not a match the empty fields will get the value NULL.

5.CROSS JOIN - The CROSS JOIN keyword matches ALL records from the left table with EACH record from the right table. This produces the Cartesian product of the two tables — i.e., every combination of rows from both tables.

```
SELECT *
FROM cars, car_stock;
```

UNIONS

The UNION operator is used to combine the rows of two or more queries without duplicates. The queries in the union must follow these rules:

- They must have the same number of columns
- The columns must have the same data types
- The columns must be in the same order

```
SELECT id FROM cars
UNION
SELECT car_id FROM car_stock;
```

2. UNION ALL - If you want to include duplicates

```
SELECT id FROM cars
union ALL
SELECT car_id FROM car_stock;
```

WINDOW FUNCTIONS

They perform calculations and operations across a set of rows that are related to the current row without collapsing rows like group by functions.

COMMON WINDOW FUNCTIONS

ROW_NUMBER() – gives unique sequential number to each row

RANK() - ranks rows with gaps on ties

DENSE_RANK() – ranks rows without gaps on ties

AVG(), SUM() – performs calculations across rows

LAG() – shows values in previous row

LEAD() – shows values in next row

```
SELECT car_name,price,
 LAG(price) OVER (ORDER BY price) AS previous_price,
 LEAD(price) OVER (ORDER BY price) AS next_price
 FROM cars;
```

```
SELECT car_name,type,price,
 AVG(price) OVER (PARTITION BY type) AS avg_price_per_type
 FROM cars;
```

COMMON TABLE EXPRESSIONS(CTEs)

A CTE (Common Table Expression) is a temporary result set that you can reference within a SQL query. It makes complex queries easier to read, reusable, and often more efficient.

```
WITH high_stock AS (
     SELECT car_id, stock
     FROM car_stock
     WHERE stock > 5
 )
 SELECT c.car_name, hs.stock
 FROM high_stock hs
 JOIN cars c ON c.id = hs.car_id;
```

EXTRACTING DATES

1. Extrcting parts of a date

```
SELECT last_updated,
 EXTRACT(YEAR FROM last_updated) AS year,
 EXTRACT(MONTH FROM last_updated) AS month,
 EXTRACT(DAY FROM last_updated) AS day
 FROM car_stock;
```

2. Using DATE_TRUNC for Grouping by Time

```sql
SELECT DATE_TRUNC('month', last_update) AS month,
COUNT(*) AS updates
FROM car_stock
GROUP BY month
ORDER BY month;
```

3. Formatting Dates as Text

```sql
SELECT
    TO_CHAR(last_update, 'YYYY-MM-DD') AS formatted_date,
    TO_CHAR(last_update, 'Month') AS month_name,
    TO_CHAR(last_update, 'YYYY') AS year_str
FROM car_stock;
```

4. Using INTERVAL

```sql
SELECT *
FROM car_stock
WHERE last_update >= CURRENT_DATE - INTERVAL '30 days';
```